

HARNESSWAM: BRIDGING PREDICTION AND DELIBERATION IN WORLD ACTION MODELS

Zhaopeng Gu^{1,2} Bingke Zhu^{1,2*} Tianxi Lin^{3,4} Guibo Zhu^{1,2} Yingying Chen^{1,2} Kai Wang^{1,3}
Tingyu Yuan^{1,2} Chaoyang Zhao¹ Zhaowen Li^{3*} Peng Su³ Jinqiao Wang^{1,2,5}

¹ Institute of Automation, Chinese Academy of Sciences, Beijing, China

² School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing, China

³ Yinwang Intelligent Technology Co., Ltd., Shenzhen, China

⁴ Beijing Institute of Technology, Beijing, China

⁵ Wuhan AI Research, Wuhan, China

ABSTRACT

World Action Models (WAMs) jointly learn environmental dynamics and robot actions, introducing priors over physical evolution into embodied control. However, finite-horizon prediction and action generation are insufficient for complex embodied tasks that require global planning, cross-stage state maintenance, execution verification, and failure recovery. We refer to this mismatch as the *prediction-deliberation gap* of WAMs. To address this gap, we propose **HarnessWAM**, an agentic framework for WAMs. HarnessWAM employs a vision-language-model-based Task Manager to maintain an evidence-grounded scene belief and a structured task graph. A capability-conditioned executable-space projection further constrains open-ended semantic plans into sequences of atomic skills that satisfy task dependencies, embodiment-state constraints, and the capability boundary of the underlying WAM. During execution, HarnessWAM operates through an event-driven, dual-timescale feedback loop: a lightweight progress estimator continuously provides high-frequency execution evidence, while the Task Manager deliberates at salient milestones by jointly considering the current observation, task state, and interaction history to determine whether to advance the task, acquire additional observations, revise the plan, or initiate local recovery. This mechanism enables the robot to recover its state after a subtask failure and resume execution without discarding previously acquired scene knowledge. HarnessWAM achieves state-of-the-art full-task and subtask success rates of 59.6% and 69.9% on RoboMemArena, and an SR of 23.7% on RoboCerebra Ideal. These results demonstrate that model-external structured state maintenance and closed-loop agentic decision making can effectively extend the local control capabilities of WAMs into embodied task execution that is plannable, verifiable, and recoverable.

1 INTRODUCTION

World Action Models (WAMs) jointly model future observations, environment states, and robot actions, enabling control policies to learn environmental dynamics and the physical consequences of robotic interventions (Li et al., 2026; Ye et al., 2026). Their predictive representations provide a foundation for outcome-aware control, closed-loop correction, and generalizable embodied agents.

However, finite-horizon prediction alone is insufficient for persistent agentic decision making. Although WAMs can reliably predict near-term physical evolution and execute local manipulation skills, they do not explicitly maintain the global task state required to verify outcomes, propagate evidence across stages, or determine how to recover from failure. We call this mismatch between local predictive control and the persistent deliberation required by open-ended embodied tasks the *prediction-deliberation gap*. Figure 1 illustrates this gap in a partially observable task whose target is revealed only through sequential exploration.

Language-model agents demonstrate that a foundation model’s effective capabilities depend on the harness through which it maintains state, invokes actions, and incorporates feedback (Yao et al., 2022;

*Corresponding authors.

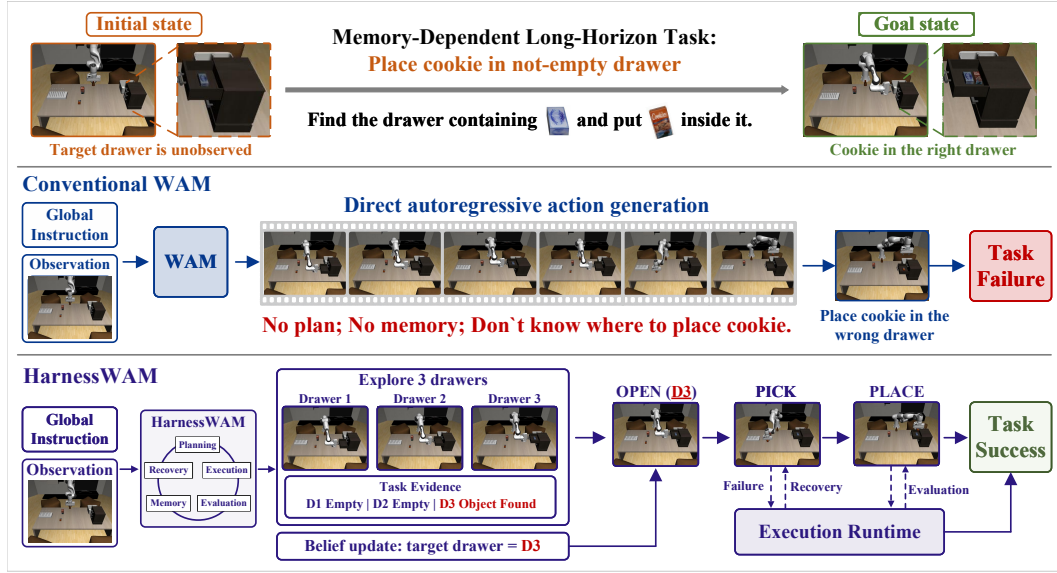


Figure 1: Prediction–deliberation gap in a memory-dependent task. A conventional WAM leaves the target unresolved, whereas HarnessWAM accumulates evidence, binds the target, and coordinates subsequent execution.

Shinn et al., 2023; Yang et al., 2024). Existing harnesses mainly target digital environments with discrete interfaces and explicit feedback. Physical interaction instead involves continuous control, partial observability, uncertain outcomes, and potentially irreversible state changes. An embodied harness must consequently ground semantic reasoning in executable skills while persistently tracking task state and physical effects.

We introduce **HarnessWAM**, an agentic framework that addresses this challenge through a structured runtime external to the WAM. A vision-language-model-based *Task Manager* maintains an evidence-grounded scene belief and represents the global instruction as a task graph containing both physical and cognitive operations. Unresolved entities remain symbolic until sufficient visual evidence supports their binding. A *capability-conditioned executable-space projection* then compiles open-ended semantic plans into primitive sequences supported by validated WAM skills, enforcing task dependencies and consistency with the current scene and embodiment state.

HarnessWAM coordinates physical execution through an event-driven, dual-timescale loop. On the fast timescale, a lightweight progress estimator continuously summarizes subtask progress and completion evidence. On the slow timescale, the Task Manager deliberates only at salient milestones or changes in execution conditions, jointly considering the current observation, progress history, scene belief, task graph, and execution state. It determines whether to continue, advance, acquire evidence, replan, or recover, and may revise only the unexecuted graph suffix when new observations resolve prior uncertainty. Upon local failure, HarnessWAM restores the robot embodiment toward its subtask-initial state while retaining acquired scene knowledge and task memory. The resulting loop integrates planning, execution, evaluation, memory, and recovery without requiring changes to the WAM architecture.

HarnessWAM achieves state-of-the-art full-task and subtask success rates of 59.6% and 69.9% on RoboMemArena (Lei et al., 2026). On RoboCerebra Ideal (Han et al., 2026), it attains an SR of 23.7%. These results show that structured agentic orchestration substantially improves WAM reliability in partially observable, memory-dependent, and multi-stage manipulation.

Our main contributions are:

- We identify the prediction-deliberation gap and formulate reliable WAM execution as model-external state estimation and closed-loop decision making.

- We introduce evidence-grounded task state and capability-conditioned executable-space projection to compile open-ended VLM plans into feasible WAM skill sequences.
- We develop an event-driven, dual-timescale mechanism for progress-aware task transitions, plan revision, and local embodiment-state recovery.
- We establish state-of-the-art performance on RoboMemArena and RoboCerebra Ideal, validating HarnessWAM on challenging long-horizon embodied tasks.

2 RELATED WORK

2.1 PREDICTIVE WORLD MODELS AND WORLD ACTION MODELS

Action-conditioned visual prediction has long supported model-predictive robot control by forecasting the effects of candidate actions (Ebert et al., 2017; 2018b). Recent generative approaches scale this principle through text-conditioned video policies, pretrained video diffusion, and unified video simulation and action decoding (Du et al., 2023; Wen et al., 2024; Liao et al., 2025). These results establish future visual modeling as a useful source of physical-dynamics priors for policy learning.

World Action Models (WAMs) more directly couple future-observation modeling with action generation. LingBot-VA enables closed-loop asynchronous control through autoregressive video–action generation (Li et al., 2026), while DreamZero demonstrates transfer across tasks, scenes, and embodiments (Ye et al., 2026). Fast-WAM separates video co-training from test-time future generation, showing that representations acquired through predictive training can support efficient action inference without explicit rollout synthesis (Yuan et al., 2026). Recent extensions additionally incorporate compressed or boundary-triggered history into WAM inference (Sun et al., 2026; Yang et al., 2026). This line of work primarily advances dynamics representation, transfer, memory conditioning, and inference efficiency within the model. HarnessWAM addresses an orthogonal question: how to organize local WAM predictions and actions into persistent task-level behavior whose intermediate effects can be verified and whose failures can be recovered.

2.2 HARNESSES FOR EMBODIED AGENTS

Language-agent research demonstrates that foundation-model capability depends strongly on the external loop through which the model maintains state, invokes actions, and incorporates feedback (Yao et al., 2022; Shinn et al., 2023; Yang et al., 2024). Such harnesses are commonly developed for software environments with discrete interfaces, explicit return values, and repeatable execution. Physical interaction instead involves continuous control, partial observability, uncertain action outcomes, and state changes that may be irreversible. Consequently, an embodied harness must ground semantic decisions in physically executable skills and maintain task state beyond individual model calls.

Robot planning provides several components of this interface. SayCan grounds language plans with skill affordances (Ahn et al., 2022); Inner Monologue introduces environment feedback into language planning (Huang et al., 2022); and SayPlan and VoxPoser ground high-level reasoning in scene geometry (Rana et al., 2023; Huang et al., 2023). Task-and-motion planning and manipulation primitives similarly connect symbolic structure to continuous feasibility (Garrett et al., 2021; Morrow & Khosla, 1997; Chen et al., 2024). Complementary work studies persistent embodied memory (Guhur et al., 2023; Shi et al., 2026; Blukis et al., 2022; Dai et al., 2026), stage-aware progress and outcome modeling (Maeda et al., 2020; Agia et al., 2024; Luo, 2024; Chen et al., 2025), and failure detection or corrective execution (Ebert et al., 2018a; Zhao et al., 2026; Ying et al., 2026).

HarnessWAM unifies these ingredients around a WAM executor. A Task Manager maintains an evidence-grounded belief and structured task graph, while a deterministic projection compiles open-ended semantic plans into validated WAM skills subject to task dependencies and embodiment constraints. Progress estimation and sparse semantic verification govern task transitions, and embodiment-state recovery handles local execution failures without discarding acquired scene knowledge. This formulation makes planning, memory, verification, and recovery components of a single event-driven decision process rather than independent additions to the control policy.

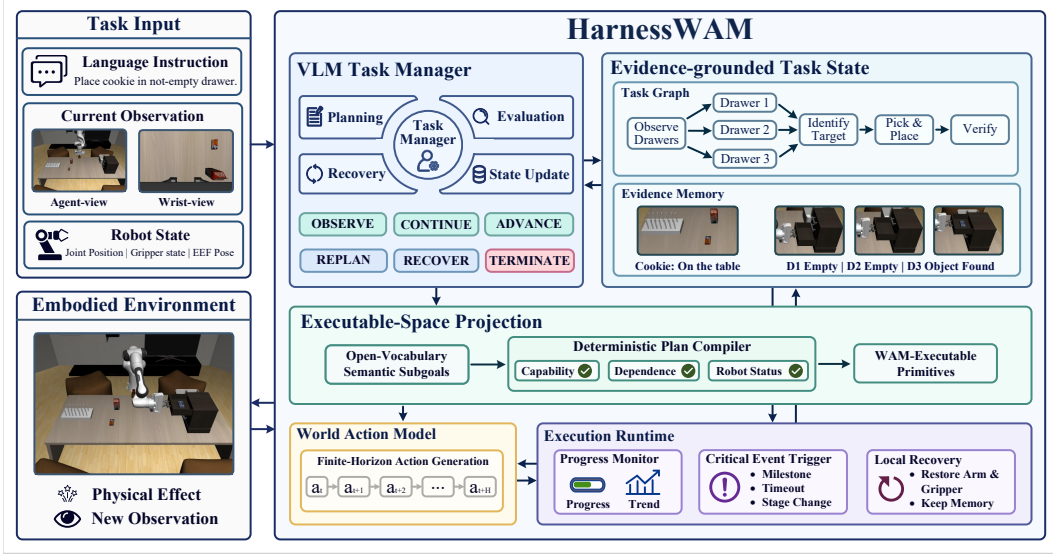


Figure 2: Overview of HarnessWAM. The VLM Task Manager converts a global instruction and current observations into an evidence-grounded task graph. Executable-space projection compiles open-vocabulary semantic subgoals into WAM-supported primitives subject to capability, dependency, and embodiment-state constraints. The WAM and execution runtime form a fast control loop, while event-triggered Task Manager decisions update state, revise the plan, recover the robot embodiment, or terminate execution.

3 METHOD

3.1 PROBLEM FORMULATION

Consider an embodied manipulation task specified by a natural-language instruction x . At time t , the environment returns multi-view RGB observations $o_t = (I_t^{\text{agent}}, I_t^{\text{wrist}})$ and the robot proprioceptive state q_t . Given a local skill instruction g_k , a World Action Model W_θ generates an action chunk of horizon H conditioned on a finite interaction history:

$$A_t = W_\theta(o_{\leq t}, q_t, g_k) = (a_t, \dots, a_{t+H-1}). \quad (1)$$

Each WAM invocation solves a local, finite-horizon control problem defined by g_k . Completing the global task additionally requires maintaining latent state across execution stages, assessing physical effects, and revising subsequent decisions as new evidence arrives. We therefore define a *harness* as a model-external decision process $\mathcal{H}$ operating over discrete task events. It transforms a global instruction into a sequence of verified WAM invocations with the objective of maximizing task-level success. At event time τ_k , we define the embodied runtime state as

$$z_k = (B_k, G_k, M_k, r_k), \quad (2)$$

where B_k , G_k , M_k , and r_k denote the scene belief, structured task graph, task memory, and execution state of the active skill, respectively. HarnessWAM recursively updates this state from a new observation and event e_k , and selects the next local goal:

$$(z_{k+1}, g_{k+1}) = \mathcal{H}(x, z_k, o_{\tau_k}, e_k). \quad (3)$$

This formulation separates continuous WAM control from task-level deliberation. The WAM realizes a local skill, while the harness determines which skill to invoke, when its execution should terminate, how its outcome changes the task state, and how execution should proceed after failure. The overall objective is to maximize goal satisfaction at task termination.

Figure 2 summarizes the HarnessWAM architecture and its event-driven interaction with the embodied environment.

3.2 EVIDENCE-GROUNDED TASK STATE

In a partially observable environment, the current image is generally insufficient for selecting future behavior. Once a drawer has been closed, for example, the current observation no longer reveals whether it contained a target object. HarnessWAM therefore maintains a scene belief B_k comprising entities together with their attributes, relations, and epistemic status. Each scene fact is represented as

$$f = (s, p, o, v, \eta, c, \mathcal{E}), \quad (4)$$

where s , p , o , and v denote the subject, predicate, object, and value; $\eta \in \{\text{observed}, \text{inferred}, \text{unknown}\}$ distinguishes direct observations, inferred facts, and unresolved state; c is a confidence score; and $\mathcal{E}$ references the supporting RGB evidence. This representation distinguishes *not observed* from *observed to be false*: an object becoming occluded does not invalidate a previously established scene fact.

To preserve the cross-stage history and visual evidence underlying this belief, HarnessWAM maintains

$$M_k = (M_k^{\text{task}}, M_k^{\text{evidence}}), \quad (5)$$

where M_k^{task} stores completed and failed nodes, retry counts, variable bindings, and the plan-revision history, while M_k^{evidence} stores visual evidence associated with salient task events. The Task Manager invokes the VLM only at event times to jointly update the scene belief and memory:

$$(B_{k+1}, M_{k+1}) = \mathcal{U}_{\text{VLM}}(x, o_{\tau_k}, B_k, M_k, e_k). \quad (6)$$

Long interaction histories are thus compressed into a queryable and updateable task state whose claims can be traced to visual observations. This explicit compression reduces the need for a VLM to reconstruct task history implicitly from a long video context.

The Task Manager further represents the task as a directed graph

$$G_k = (V_k, E_k, \mathcal{X}_k, \beta_k), \quad (7)$$

where V_k is the node set, E_k contains dependency edges, $\mathcal{X}_k$ comprises unresolved symbolic variables, and $\beta_k : \mathcal{X}_k \rightarrow \mathcal{O}$ is a partial binding from variables to scene entities. Each node is represented as

$$v_i = (\text{op}_i, \text{arg}_i, \text{pre}_i, \text{eff}_i, \text{term}_i, \text{rec}_i), \quad (8)$$

specifying an operation, typed arguments, preconditions, expected effects, termination conditions, and a recovery strategy. The graph includes both *motor nodes*, which induce physical actions, and *cognitive nodes*, which acquire observations, verify state, bind variables, or update memory. When an entity cannot be identified from the initial observation, the Task Manager retains a symbolic variable instead of committing to an unsupported guess, and updates β_k once sufficient evidence becomes available. This graph expresses sequential dependencies, information acquisition, conditional branches, and delayed decisions after exploration within a unified representation.

3.3 CAPABILITY-CONDITIONED EXECUTABLE-SPACE PROJECTION

A VLM can propose plans in an open semantic space, whereas a WAM can reliably execute only those local skills supported by its training distribution and control interface. HarnessWAM connects these levels through a projection from open-ended semantic plans to a capability-conditioned executable space, after which the WAM produces continuous control commands.

We define an extensible ontology of parameterized operations:

$$\mathcal{P}^* = \mathcal{P}_{\text{motion}} \cup \mathcal{P}_{\text{grasp}} \cup \mathcal{P}_{\text{contact}} \cup \mathcal{P}_{\text{articulation}} \cup \mathcal{P}_{\text{assembly}} \cup \mathcal{P}_{\text{tool}}, \quad (9)$$

corresponding to free-space motion, object acquisition and release, contact-rich interaction, articulated-object manipulation, assembly, and tool use. Each primitive denotes an operation family with open object arguments and explicit physical semantics, while the WAM generates a concrete trajectory from the current observation. A primitive has the unified representation

$$p = (\tau_p, \Theta_p, \text{Pre}_p, \text{Eff}_p, \text{Term}_p, \text{Rec}_p), \quad (10)$$

where τ_p denotes the interaction type, Θ_p is a typed parameter space, and the remaining terms specify preconditions, expected effects, termination criteria, and recovery rules.

This compact interface is motivated by empirical evidence that motor behavior exhibits low-dimensional, repetitive, and compositional structure. In natural work activities, the ten most frequent human grasp types account for 81% of grasp duration and 72% of grasp instances (Bullock et al., 2013). Two kinematic primitives explain 95% of the cumulative variance in discrete reaching motions (Moro et al., 2012), and two postural hand synergies explain more than 80% of the variance in 15-DoF grasps over 57 objects (Santello et al., 1998). Robotic manipulation can likewise be organized through a finite set of relative-motion classes between rigid bodies (Morrow & Khosla, 1997). These findings motivate a compact, parameterized, and extensible behavior basis for organizing high-dimensional control. The ontology can grow with the validated capabilities of the WAM and the requirements of the task domain.

For a particular WAM, the executable primitive set is determined by the skills that the model has acquired and that have been empirically validated:

$$\mathcal{P}_W = \{p \in \mathcal{P}^* \mid p \text{ has a validated realization under } W_\theta\}. \quad (11)$$

Let $\mathcal{L}(\mathcal{P}_W)$ denote the plan language generated by supported primitives, and let $\mathcal{F}(z_k)$ denote the feasible set induced by the current scene, object bindings, task dependencies, and embodiment state. HarnessWAM projects a VLM-generated graph as

$$G_k^{\text{exec}} = \Pi_{\mathcal{L}(\mathcal{P}_W) \cap \mathcal{F}(z_k)} (G_k^{\text{vlm}}). \quad (12)$$

The projection is implemented by a deterministic plan compiler. It checks argument types, node dependencies, precondition–effect consistency, single-arm holding state, and graph acyclicity, and then canonicalizes valid nodes into local WAM instructions. For example, `PLACE(object, target)` requires the same object to have been acquired, `POUR(object, target)` requires the robot to be holding that object, and `OPEN` and `CLOSE` must satisfy the corresponding gripper and object-state constraints. A semantic node that admits a composition of supported primitives is expanded into a valid sequence. If no feasible projection exists, the compiler returns $\perp$ and reports the violated constraints to the Task Manager for replanning.

New observations may change variable bindings or future branches. To preserve consistency with the physical history, HarnessWAM permits revisions only to the unexecuted suffix of the task graph and enforces the history-invariance constraint

$$G_{k+1}[V_k^{\text{executed}}] = G_k[V_k^{\text{executed}}]. \quad (13)$$

3.4 PROGRESS-CONDITIONED EVENT CONTROL

Invoking a VLM at every environment step is computationally expensive and exposes task-level decisions to transient visual fluctuations. HarnessWAM instead adopts dual-timescale control: the WAM and a lightweight progress estimator form a fast execution loop, while the Task Manager forms a slow deliberation loop triggered by semantic events.

In the fast loop, the WAM repeatedly generates action chunks conditioned on the local instruction g_k of the active node. A prompt-conditioned progress estimator F_ϕ predicts continuous progress and completion likelihood from the most recent L frames of dual-view RGB observations and the text:

$$(p_t, c_t, \pi_t^{\text{bin}}) = F_\phi(o_{t-L+1:t}, g_k), \quad (14)$$

where $p_t \in [0, 1]$ denotes continuous progress, $c_t \in [0, 1]$ denotes the probability of stage completion, and π_t^{bin} is a discrete distribution over progress intervals. F_ϕ extracts multi-view spatial features with a frozen vision–language encoder and models local temporal changes with a causal temporal module. It is trained with

$$\mathcal{L}_{\text{prog}} = \lambda_r \mathcal{L}_{\text{reg}} + \lambda_b \mathcal{L}_{\text{bin}} + \lambda_r \mathcal{L}_{\text{rank}} + \lambda_e \mathcal{L}_{\text{endpoint}} + \lambda_s \mathcal{L}_{\text{success}} + \lambda_m \mathcal{L}_{\text{mono}}, \quad (15)$$

whose terms supervise continuous progress, progress intervals, temporal ordering, trajectory endpoints, stage completion, and local monotonicity, respectively. The estimator continuously supplies execution evidence for the active skill. The runtime converts the progress sequence into candidate milestone events; exhaustion of a skill budget or changes in a condition or variable binding also trigger task-level deliberation. Progress predictions alone never advance the task graph.

Algorithm 1 Event-driven HarnessWAM inference

Require: Instruction x , initial RGB observation o_0 , WAM W_θ , task budget Ω
Ensure: Task outcome in $\{\text{success}, \text{failure}\}$

```

1:  $(B, M) \leftarrow \text{INITIALIZE}(x, o_0)$ 
2:  $G^{\text{vlm}} \leftarrow \text{PLAN}(x, B, M)$ 
3:  $G \leftarrow \text{PROJECT}(G^{\text{vlm}}, W_\theta, B)$ 
4: while  $\Omega > 0$  and  $G$  contains an unfinished required node do
5:    $v \leftarrow \text{SELECTREADYNODE}(G)$ 
6:   if  $v$  is a cognitive node then
7:     Acquire evidence and update  $B, M$ , bindings, and the graph suffix
8:     Mark  $v$  complete and reproject the unexecuted graph
9:     continue
10:  end if
11:   $(q^0, u^0) \leftarrow$  current arm and gripper state; activate  $v$ 
12:  while  $v$  is active and  $\Omega > 0$  do
13:     $A_t \leftarrow W_\theta(o_{\leq t}, q_t, \text{PROMPT}(v))$ 
14:    Execute  $A_t$ , observe  $o_{t+1}$ , and update  $\Omega$ 
15:     $\hat{s}^{\text{prog}} \leftarrow F_\phi(\text{recent RGB}, \text{PROMPT}(v))$ 
16:    if  $\text{EVENT}(\hat{s}^{\text{prog}}, r, G)$  then
17:       $(\nu, d, \Delta B, \Delta \beta, \rho) \leftarrow \mathcal{T}_{\text{VLM}}(x, o_{t+1}, B, G, M, r, \hat{s}^{\text{prog}})$ 
18:      Update  $B, M$ , bindings, and, if  $\rho$ , the unexecuted graph suffix
19:      if  $d = \text{advance}$  then
20:        Mark  $v$  complete; break
21:      else if  $d = \text{observe}$  then
22:        Acquire additional evidence and update the task state
23:      else if  $d = \text{replan}$  then
24:        Revise and reproject the graph suffix; deactivate  $v$ 
25:      else if  $d = \text{recover}$  then
26:        Restore  $(q^0, u^0)$  and clear the local WAM state
27:        Revise and project the recovery plan; deactivate  $v$ 
28:      else if  $d = \text{terminate}$  then
29:        return failure
30:      end if
31:    end if
32:  end while
33: end while
34: if all required nodes are complete and  $\text{VERIFYGOAL}(x, o, B)$  then
35:   return success
36: else
37:   return failure
38: end if
    
```

At a candidate event e_k , the Task Manager jointly reasons over the current RGB observations, progress-estimate history, scene belief, task graph, historical evidence, and execution state:

$$\begin{aligned}
 y_k &= (\nu_k, d_k, \Delta B_k, \Delta \beta_k, \rho_k) \\
 &= \mathcal{T}_{\text{VLM}}(x, o_{\tau_k}, B_k, G_k, M_k, r_k, \hat{s}_k^{\text{prog}}, e_k),
 \end{aligned} \tag{16}$$

where $\hat{s}_k^{\text{prog}}$ summarizes recent progress, completion likelihood, and their temporal trends. The outcome label $\nu_k \in \{\text{success}, \text{failure}, \text{uncertain}\}$ characterizes the physical effect, and $d_k \in \{\text{continue}, \text{advance}, \text{observe}, \text{replan}, \text{recover}, \text{terminate}\}$ is the execution decision. ΔB_k and $\Delta \beta_k$ update the scene belief and variable bindings, respectively, while ρ_k indicates whether the unexecuted plan should be revised. The active node is marked complete and its successors are enabled only when $d_k = \text{advance}$. The progress estimator thus provides high-frequency execution cues, while the Task Manager determines subtask boundaries from visual outcomes and the structured task state maintained throughout execution.

3.5 EMBODIMENT-STATE RECOVERY AND TASK TERMINATION

Physical execution deviations, perceptual uncertainty, and plan-level failures require different responses. At the beginning of each motor node, HarnessWAM records the arm joint state q_k^0 and gripper state u_k^0 , and assigns the node an execution budget T_k . Budget exhaustion or a verified failure emits a failure-handling event. At this event, the Task Manager reasons over the current RGB observation, progress trajectory, scene belief, task graph, attempt history, and remaining task-level budget encoded in r_k . It may continue a slowly progressing skill, acquire additional evidence when the outcome is ambiguous, recover from a local execution deviation, revise the unexecuted plan when the current strategy is invalid, or terminate when no feasible continuation remains.

When $d_k = \text{recover}$, the saved embodiment state provides a physically grounded recovery target. Multi-step joint control drives the arm and gripper toward (q_k^0, u_k^0) :

$$(q_t, u_t) \xrightarrow[\text{multi-step control}]{\text{recover}} (q_k^0, u_k^0). \quad (17)$$

Recovery resets only the robot embodiment, preserving the environment, scene belief, and information acquired during prior execution. The local WAM state is then cleared. The Task Manager may retry the active node with a revised local goal or replace the unexecuted graph suffix with an alternative strategy, after which the resulting plan is projected back into the executable space. The outcome of each attempt is recorded in M_k , allowing subsequent recovery decisions to depend on accumulated evidence and prior failures rather than a fixed per-node retry count.

The harness determines task termination dynamically, allowing trajectory length to follow physical progress. A task succeeds when every required node is complete and final visual verification confirms the global goal. It fails when the graph terminates without satisfying the goal, the Task Manager determines that no plan supported by the available WAM skills remains feasible, or the bounded task-level execution and recovery budget is exhausted. A finite task graph and bounded task-level budget guarantee eventual termination.

3.6 OVERALL INFERENCE PROCEDURE

Algorithm 1 summarizes the complete inference procedure. This procedure characterizes HarnessWAM as an event-driven recursive state-estimation and decision system. Structured memory supplies cross-stage state, executable-space projection constrains VLM decisions, progress estimation connects continuous control to discrete task events, and local recovery handles physical execution deviations. Together, these components organize finite-horizon WAM invocations into persistent, verifiable, and recoverable embodied-agent behavior.

4 EXPERIMENTS

We evaluate whether model-external agentic orchestration improves the reliability of WAM-based embodied task execution across memory-dependent and long-horizon compositional settings. In addition to full-task and subtask success, controlled ablations and plan-level diagnostics isolate how structured task state, executable-space projection, event-driven control, and failure recovery contribute to the resulting behavior.

4.1 EXPERIMENTAL SETUP

Benchmarks. We evaluate HarnessWAM on RoboMemArena (Lei et al., 2026) and RoboCerebra Ideal (Han et al., 2026). RoboMemArena comprises 26 long-horizon manipulation tasks with an average trajectory length of 1,076 environment steps, and 68.9% of its subtasks depend on historical information. The benchmark contains four complementary task families: multi-object transfer, occlusion, counting, and sequential execution, with 4, 11, 7, and 4 tasks, respectively. These families evaluate persistent tracking of completed operations, maintenance of occluded scene state, repeated-action counting, and cross-stage reference resolution. RoboCerebra targets long-horizon compositional manipulation and high-level reasoning. The two benchmarks are complementary: RoboMemArena emphasizes history-dependent decisions under partial observability, whereas RoboCerebra Ideal emphasizes reliable extended-plan generation and execution.

Metrics. For RoboMemArena, we report full-task success and subtask success. Let task i contain K_i stage-level verification predicates, where $\psi_i^{(k)}$ indicates whether the goal state of subtask k is satisfied. Full-task success is defined as

$$\text{SR}_{\text{task}} = \frac{1}{N} \sum_{i=1}^N \prod_{k=1}^{K_i} \mathbb{I}[\psi_i^{(k)} = 1]. \quad (18)$$

Subtask success is the macro-average fraction of completed subtasks:

$$\text{SR}_{\text{sub}} = \frac{1}{N} \sum_{i=1}^N \frac{1}{K_i} \sum_{k=1}^{K_i} \mathbb{I}[\psi_i^{(k)} = 1]. \quad (19)$$

The former measures end-to-end reliability, while the latter retains information about partial progress when the full task is not completed. For RoboCerebra, we follow the official protocol and compute SR as the mean completion rate of key object-state transitions:

$$\text{SR}_{\text{RC}} = \frac{1}{N} \sum_{i=1}^N \frac{1}{K_i} \sum_{k=1}^{K_i} \mathbb{I}[\psi_i^{(k)} = 1], \quad (20)$$

where K_i is the number of key object-state transitions in task i . We report this metric on the RoboCerebra Ideal subset.

Evaluation protocol. For HarnessWAM and the same-WAM diagnostic variants, each task is evaluated over 20 rollouts with matched initial states, random seeds, observation interfaces, and task-level execution budgets. We report macro-averages across tasks and rollouts. Published baseline numbers are taken from the corresponding benchmark evaluations. HarnessWAM determines subtask transitions and episode termination dynamically from execution evidence within the task-level budget; it does not use predetermined switching times.

Models and implementation details. We use LingBot-VA as the underlying WAM (Li et al., 2026). Its architecture is unchanged and is fine-tuned separately on the training data of each benchmark. Within a benchmark, all comparisons and ablations share the same WAM checkpoint, isolating the contribution of task-level orchestration. On RoboMemArena, the WAM receives 256×256 agent-view and wrist-view RGB images together with robot state, and generates action chunks conditioned on a local skill instruction. The Task Manager uses Qwen3-VL-32B-Instruct without task-specific fine-tuning. Its visual input consists only of multi-view RGB, without depth, segmentation labels, or privileged simulator state.

The progress estimator takes the latest five timesteps of dual-view RGB and the active skill instruction as input. A frozen *SigLIP2-base-patch16-256* encoder extracts multi-view spatial features, followed by a four-layer causal Transformer that models local temporal evolution. Its objective combines continuous progress regression, interval classification, pairwise ranking, endpoint anchoring, completion prediction, and local monotonicity. We split training and validation data by episode and select the checkpoint with the lowest validation progress error.

Baselines. On RoboMemArena, we compare against $\pi_{0.5}$ (Intelligence et al., 2025), HiF-VLA (Lin et al., 2026), MemoryVLA (Shi et al., 2026), MemER (Sridhar et al., 2026), and PrediMem (Lei et al., 2026) baselines. We additionally construct two diagnostic baselines using the same LingBot-VA checkpoint as HarnessWAM. **WAM + Whole Task** conditions the WAM directly on the global instruction, without explicit decomposition or persistent task state. **WAM + Static Plan** generates a linear subtask sequence once at initialization and holds it fixed throughout execution, without memory updates or replanning. On RoboCerebra Ideal, we compare against $\pi_{0.5}$ (Intelligence et al., 2025), OpenVLA (Kim et al., 2024), GPT-4o Planner + OpenVLA, and the HPE Framework (Han et al., 2026). All HarnessWAM variants use Qwen3-VL-32B-Instruct as the Task Manager.

4.2 MAIN RESULTS

RoboMemArena. Table 1 compares full-task and subtask success across the four task families. The published methods provide benchmark-level context, while WAM + Whole Task and WAM + Static Plan control for the LingBot-VA checkpoint and isolate the effect of task-level orchestration.

Table 1: Full-task and subtask success rates on RoboMemArena (%).

Method	Transfer		Occlusion		Counting		Sequence		Average	
	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}
<i>(a) Published baselines</i>										
$\pi_{0.5}$	20.0	42.8	12.7	17.2	14.3	50.9	60.0	71.6	21.5	38.7
HiF-VLA	17.5	38.9	12.7	27.1	8.6	45.9	42.5	70.2	16.9	39.8
MemoryVLA	15.0	37.2	7.3	13.1	14.3	55.1	37.5	65.2	15.0	35.3
MemER	20.0	36.1	16.4	33.2	27.1	65.1	65.0	79.1	27.3	49.1
PrediMem	22.5	45.2	27.3	38.4	45.7	69.3	72.5	89.5	38.5	55.2
<i>(b) WAM-based variants</i>										
WAM + Whole Task	11.3	12.5	41.3	42.8	51.4	75.8	73.8	77.1	44.4	52.3
WAM + Static Plan	15.0	16.7	38.2	49.8	67.2	90.1	73.8	91.9	47.9	62.0
HarnessWAM	21.3	31.5	55.0	63.9	73.6	88.2	86.3	93.0	59.6	69.9

Table 2: Success rate on RoboCerebra Ideal (%).

Method	Ideal SR $\uparrow$
$\pi_{0.5}$	1.88
OpenVLA	7.84
GPT-4o Planner + OpenVLA	21.92
HPE Framework	21.10
HarnessWAM	23.70

HarnessWAM achieves the best average performance under both metrics, reaching 59.6% full-task success and 69.9% subtask success. These results exceed PrediMem by 21.1 and 14.7 percentage points, respectively. Within the controlled WAM comparison, introducing a static decomposition improves WAM + Whole Task by 3.5 points in full-task success and 9.7 points in subtask success. HarnessWAM adds a further 11.7 and 7.9 points over WAM + Static Plan, showing that initial decomposition alone does not account for the improvement; persistent task state and closed-loop task management remain necessary for reliable composition.

The task-family breakdown further separates local skill reliability from successful composition. Relative to WAM + Static Plan, HarnessWAM improves full-task success by 16.8 points on occlusion and 12.5 points on sequential execution, while its largest subtask gain is 14.8 points on transfer. On counting, subtask success decreases by 1.9 points, yet full-task success increases by 6.4 points, indicating that HarnessWAM more consistently composes locally completed stages into a correct end-to-end execution.

RoboCerebra. Table 2 reports performance on the Ideal subset. HarnessWAM achieves an SR of 23.70%, outperforming GPT-4o Planner + OpenVLA by 1.78 percentage points and the HPE Framework by 2.60 points. Because RoboCerebra Ideal is static and fully observable, this improvement shows that the benefits of HarnessWAM extend beyond explicit memory recovery: dependency-aware planning, outcome-conditioned transitions, and failure-aware adaptation also improve the execution of extended multi-skill plans.

4.3 ABLATION STUDIES

We perform all ablations on RoboMemArena using identical LingBot-VA weights, initial states, and evaluation seeds. Table 3 studies five interventions. **Without task state** removes structured scene facts, interaction history, and variable bindings, leaving the Task Manager with only the current image and global instruction. **Without executable projection** bypasses capability, argument, precondition, holding-state, and dependency checks on the VLM plan. **Without progress events** replaces progress-conditioned event triggering with fixed-interval Task Manager invocation. **Progress-only switching** allows the progress estimator to determine subtask completion without semantic outcome verification.

Table 3: Ablation results on RoboMemArena (%).

Method	Transfer		Occlusion		Counting		Sequence		Average	
	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}	SR _{task}	SR _{sub}
HarnessWAM	21.3	31.5	55.0	63.9	73.6	88.2	86.3	93.0	59.6	69.9
w/o task state	15.1	16.9	38.4	49.1	66.2	87.1	73.8	91.9	47.7	61.1
w/o executable projection	5.0	10.6	9.1	29.3	23.6	54.8	48.8	61.7	18.5	38.3
w/o progress events	13.8	17.5	37.3	43.1	41.4	70.9	60.0	69.6	38.3	50.7
progress-only switching	26.3	29.6	53.1	61.7	65.7	85.0	72.5	97.9	55.4	68.4
w/o recovery	21.2	33.5	51.3	61.2	65.7	86.9	75.0	85.8	54.2	67.7

Table 4: Effect of executable-space projection on plan quality (%).

Plan representation	Syntax ↑	Dependencies ↑	Binding ↑	Executability ↑
Raw VLM plan	60.8	58.1	21.3	13.8
+ normalization and aliasing	84.6	67.5	63.8	42.3
+ executable-space projection	95.2	92.9	88.3	72.9

Without recovery terminates after a detected failure or budget exhaustion, rather than restoring the embodiment state and adapting the remaining plan.

Task-level effects. Executable-space projection has the largest measured contribution: removing it reduces average full-task success from 59.6% to 18.5% and subtask success from 69.9% to 38.3%, corresponding to drops of 41.1 and 31.6 percentage points. The degradation spans all four task families, indicating that a semantically plausible plan is often insufficient unless its operators, arguments, dependencies, and embodiment-state transitions conform to the WAM execution interface. Removing progress-conditioned events produces the next largest decline, lowering the two metrics by 21.3 and 19.2 points; fixed-frequency deliberation therefore provides a poor substitute for execution-aware event selection.

Removing structured task state yields 47.7% full-task and 61.1% subtask success, a decrease of 11.9 and 8.8 points from HarnessWAM. Its largest full-task degradation occurs on occlusion, consistent with the need to preserve evidence after relevant scene content becomes hidden. Progress-only switching retains a similar average subtask success (68.4% versus 69.9%) but lowers full-task success to 55.4%. The contrast is particularly pronounced on sequential execution, where subtask success rises to 97.9% while full-task success falls from 86.3% to 72.5%; high local completion scores therefore do not substitute for semantic verification of task transitions. Finally, removing recovery decreases average full-task and subtask success by 5.4 and 2.2 points, with the largest loss on sequential execution, where an unrecovered local failure can invalidate a long remaining suffix.

Plan-level diagnosis of executable-space projection. The large task-level degradation caused by removing projection motivates a direct analysis of the intermediate plans. We compare the graph generated directly by the VLM, a lexically normalized graph in which operator expressions and safe entity aliases are mapped to the canonical WAM prompt vocabulary, and the fully projected graph after capability, dependency, binding, and embodiment-state constraints are enforced. We measure syntactic validity, dependency satisfaction, object-binding accuracy, and executability. Reference decompositions are used only for this offline node- and dependency-level analysis and are never provided to HarnessWAM during inference.

Raw VLM plans exhibit substantial discrepancies with the WAM execution interface: only 60.8% satisfy the required syntax, and their executability is 13.8%. Lexical normalization and alias resolution provide a strong first-stage correction, improving syntax by 23.8 points, object binding by 42.5 points, and executability by 28.5 points. Surface canonicalization alone nevertheless leaves dependency satisfaction at 67.5% and executability at 42.3%. Enforcing the complete executable-space projection raises these metrics to 92.9% and 72.9%, corresponding to further gains of 25.4 and 30.6 points; syntax and binding accuracy also increase to 95.2% and 88.3%. These results distinguish lexical alignment from executable plan construction: canonical vocabulary reduces semantic-interface

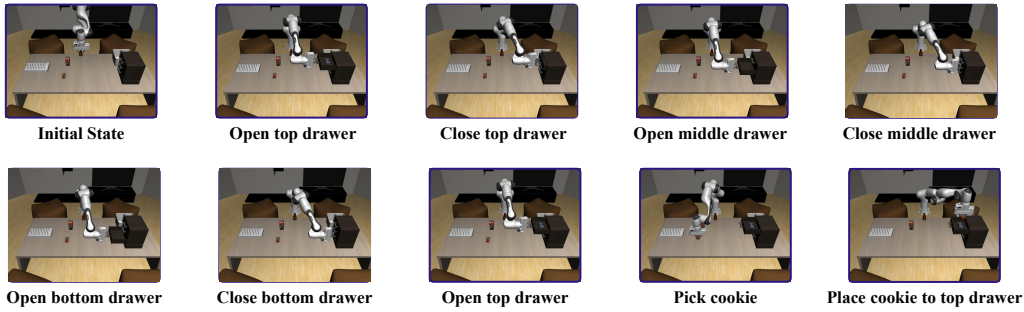


Figure 3: Selected keyframes from a representative HarnessWAM rollout on RoboMemArena Task 4. The robot opens and closes all three drawers in sequence, retains the evidence identifying the non-empty top drawer after it becomes occluded, and conditions subsequent execution on this target binding. It then reopens the target drawer and picks and places the target object inside.

mismatch, while capability, dependency, binding, and embodiment-state constraints are required to produce plans that can be reliably instantiated by the WAM. This plan-level effect is consistent with the 41.1-point decrease in average full-task success when projection is removed.

4.4 QUALITATIVE RESULTS

Figure 3 presents selected keyframes from a representative rollout on RoboMemArena Task 4. HarnessWAM sequentially opens and closes the top, middle, and bottom drawers, recording evidence about their contents whenever each drawer becomes observable. After exploration, the current RGB observation alone no longer reveals which drawer was non-empty. The retained task state nevertheless binds the target to the top drawer and instantiates the remaining manipulation sequence. HarnessWAM then reopens the top drawer, picks the target object, and places it inside. The rollout illustrates how information-gathering actions, cross-stage evidence, delayed target binding, and local WAM skills support coherent execution beyond the observable context of any individual skill.

5 CONCLUSION

We introduced HarnessWAM, a model-external agentic framework that bridges finite-horizon WAM execution and the persistent deliberation required by complex embodied tasks. HarnessWAM organizes local WAM skills through an evidence-grounded scene belief and structured task graph, constrains open-ended VLM plans through capability-conditioned executable-space projection, and couples high-frequency progress estimation with event-triggered semantic verification and embodiment-state recovery. On RoboMemArena, HarnessWAM achieves 59.6% full-task success and 69.9% subtask success; on RoboCerebra Ideal, it achieves an SR of 23.7%. Controlled comparisons and ablations show that the gains cannot be explained by the underlying WAM or an initial decomposition alone. Plan-level diagnostics further show that lexical normalization closes only part of the semantic-interface gap: enforcing capability, dependency, binding, and embodiment-state constraints raises plan executability from 42.3% to 72.9%. Together with persistent task state, execution-aware transitions, and recovery, this constrained planning interface enables more reliable multi-stage composition. These results support a broader view of WAM-based embodied intelligence in which task-level reliability emerges from the interaction between predictive skill execution and a structured agentic runtime. Extending this framework to real-world manipulation, broader WAM skill repertoires, and calibrated uncertainty-aware deliberation constitutes an important direction for future work.

REFERENCES

Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeanette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. *arXiv preprint arXiv:2410.04640*, 2024.

- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- Valts Blukis, Chris Paxton, Dieter Fox, Animesh Garg, and Yoav Artzi. A persistent spatial semantic representation for high-level natural language instruction execution. In *Conference on Robot Learning*, pp. 706–717. PMLR, 2022.
- Ian M Bullock, Joshua Z Zheng, Sara De La Rosa, Charlotte Guertler, and Aaron M Dollar. Grasp frequency and usage in daily household and machine shop tasks. *IEEE transactions on haptics*, 6(3):296–308, 2013.
- Qianzhong Chen, Justin Yu, Mac Schwager, Pieter Abbeel, Yide Shentu, and Philipp Wu. Sarm: Stage-aware reward modeling for long horizon robot manipulation. *arXiv preprint arXiv:2509.25358*, 2025.
- Zeren Chen, Zhelun Shi, Xiaoya Lu, Lehan He, Sucheng Qian, Zhenfei Yin, Wanli Ouyang, Jing Shao, Yu Qiao, Cewu Lu, et al. Rh20t-p: A primitive-level robotic dataset towards composable generalization agents. *arXiv preprint arXiv:2403.19622*, 2024.
- Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.
- Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.
- Frederik Ebert, Chelsea Finn, Alex X Lee, and Sergey Levine. Self-supervised visual planning with temporal skip connections. *CoRL*, 12(16):23, 2017.
- Frederik Ebert, Sudeep Dasari, Alex X Lee, Sergey Levine, and Chelsea Finn. Robustness via retrying: Closed-loop robotic manipulation with self-supervised learning. In *Conference on robot learning*, pp. 983–993. PMLR, 2018a.
- Frederik Ebert, Chelsea Finn, Sudeep Dasari, Annie Xie, Alex Lee, and Sergey Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. *arXiv preprint arXiv:1812.00568*, 2018b.
- Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293, 2021.
- Pierre-Louis Guhur, Shizhe Chen, Ricardo Garcia Pinel, Makarand Tapaswi, Ivan Laptev, and Cordelia Schmid. Instruction-driven history-aware policies for robotic manipulations. In *Conference on Robot Learning*, pp. 175–187. PMLR, 2023.
- Songhao Han, Boxiang Qiu, Yue Liao, Siyuan Huang, Chen Gao, Shuicheng Yan, and Si Liu. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *Advances in Neural Information Processing Systems*, 38, 2026.
- Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Thompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding, Zhipeng Zhang, Lida Huang, et al. Robomemarena: A comprehensive and challenging robotic memory benchmark. *arXiv preprint arXiv:2605.10921*, 2026.
- Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, et al. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025.
- Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. Hif-vla: Hindsight, insight and foresight through motion representation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20732–20742, 2026.
- Fiona Luo. Vision-language models for robot success detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 23750–23752, 2024.
- Guilherme Maeda, Joni Väättäinen, and Hironori Yoshida. Visual task progress estimation with appearance invariant embeddings for robot control and planning. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7941–7948. IEEE, 2020.
- Federico L Moro, Nikos G Tsagarakis, and Darwin G Caldwell. On the kinematic motion primitives (kmeps)—theory and application. *Frontiers in neurorobotics*, 6:10, 2012.
- J Daniel Morrow and Pradeep K Khosla. Manipulation task primitives for composing robot skills. In *Proceedings of International Conference on Robotics and Automation*, volume 4, pp. 3354–3359. IEEE, 1997.
- Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. *arXiv preprint arXiv:2307.06135*, 2023.
- Marco Santello, Martha Flanders, and John F Soechting. Postural hand synergies for tool use. *Journal of neuroscience*, 18(23):10105–10115, 1998.
- Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *International Conference on Learning Representations*, volume 2026, pp. 18567–18602, 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Scaling up memory for robotic control via experience retrieval. In *International Conference on Learning Representations*, volume 2026, pp. 97142–97166, 2026.
- Xiaoquan Sun, Ruijian Zhang, Chen Cao, Yihan Sun, Jiahui Chen, Zetian Xu, Bo Chen, Haijier Chen, Zhen Yang, Jiarun Zhu, et al. Himem-wam: Hierarchical memory-gated world action models for robotic manipulation. *arXiv preprint arXiv:2606.10363*, 2026.
- Youpeng Wen, Junfan Lin, Yi Zhu, Jianhua Han, Hang Xu, Shen Zhao, and Xiaodan Liang. Vid-man: Exploiting implicit dynamics from video diffusion model for effective robot manipulation. *Advances in Neural Information Processing Systems*, 37:41051–41075, 2024.

- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Sizhe Yang, Juncheng Mu, Tianming Wei, Chenhao Lu, Xiaofan Li, Linning Xu, Zhengrong Xue, Zhecheng Yuan, Dahua Lin, Jiangmiao Pang, et al. Memorywam: Efficient world action modeling with persistent memory. *arXiv preprint arXiv:2606.20562*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- Chenduo Ying, Linkang Du, Yuanchao Shu, and Peng Cheng. Robofairing: Retrieval-augmented and language grounding failure detection for vlm-enabled robotic manipulation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13188–13202, 2026.
- Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- Ganlong Zhao, Zijia Tang, Xingping Chen, Zhanghui Kuang, Ye Tian, and Guanbin Li. Flare: A failure-aware framework for autonomous correction and recovery in visual-language robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 22391–22401, 2026.